

R5.09 Virtualisation avancée

Michaël Hauspie

BUT 3

Plan du cours

1 Semaine 2

- Un mot sur la production logicielle
 - Du *Dev et Ops* au *DevOps*
 - Infrastructure as Code
 - Préambule à Terraform/OpenTofu

Un logiciel est fini quand

- ① ☐ les tests passent ?
- ② ☐ il fonctionne sur ma machine ?
- ③ ☐ on a fini la revue de code ?
- ④ ☐ il est en production et peut être utilisé ?

Un logiciel est fini quand

- ① ☐ ~~les tests passent~~
- ② ☐ ~~il fonctionne sur ma machine~~
- ③ ☐ ~~on a fini la revue de code~~
- ④ ☒ il est en production et peut être utilisé

Mise en production

- ① mettre l'infrastructure en place
- ② y faire tourner le code
- ③ s'assurer qu'il continue de fonctionner
- ④ en garantir la sécurité
- ⑤ ...

Plan du cours

1 Semaine 2

- Un mot sur la production logicielle
- Du *Dev et Ops* au *DevOps*
- Infrastructure as Code
- Préambule à Terraform/OpenTofu

Une équipe chargée du développement, et c'est tout

- implémente des fonctionnalités et rédige un bout de documentation
- package l'application
- transmet la patate chaude à une autre équipe pour la mise en production

Une équipe chargée de l'opérationnel

- Mettre en place les serveurs (physiquement)
 - ▶ les déballer, les installer dans les baies, les câbler...
- Installer le système et le configurer
- Installer les dépendances nécessaires à l'application

Ça marche !

Oui mais...

- mise à jour manuelle, lente et sujette à erreur
- des erreurs mènent à des différences de configuration, de version de dépendance...
- l'application ne marche plus en production
 - ▶ Ça marche sur ma machine !

Avec des solutions de type IaaS, les Ops ne doivent plus gérer la partie matérielle

- ils ont du temps pour rationaliser la mise en place de l'infrastructure
- au lieu de brancher des câbles, ils écrivent du « code »
 - ▶ Ansible, Docker, Terraform. . .

- Une plus forte collaboration entre les *Dev* et les *Ops*
- Tout mettre en œuvre pour *livrer* plus facilement
- La configuration de l'infrastructure peut évoluer **avec** le logiciel
- Les infrastructures de développement et de test sont les mêmes que l'infrastructure de production
 - ▶ configuration cohérente car non fait à la main

Plan du cours

1 Semaine 2

- Un mot sur la production logicielle
- Du *Dev et Ops* au *DevOps*
- **Infrastructure as Code**
- Préambule à Terraform/OpenTofu

Du code, du code, du code

On écrit du code pour définir, déployer et mettre à jour l'infrastructure

- scripts dédiés
- modèles
- outils de provisionnement d'infrastructure
- outils de gestion de configuration

Scripts dédiés

On clone le projet

```
git clone https://example.org/projet/code.git
```

On lance le script de configuration du serveur

```
cd code && ./configure-server.sh
```

Avantages

- Utilisation de langage habituel
- On peut imaginer n'importe quelle action de déploiement

Scripts dédiés

On clone le projet

```
git clone https://example.org/projet/code.git
```

On lance le script de configuration du serveur

```
cd code && ./configure-server.sh
```

Avantages

- Utilisation de langage habituel
- On peut imaginer n'importe quelle action de déploiement

Inconvénients

- Utilisation de langage habituel
- On peut imaginer n'importe quelle action de déploiement

Modèles

On prépare un modèle de système/application qu'on pourra utiliser en développement et en production

- Modèle de VM
 - ▶ Packer
 - ▶ Vagrant
- Modèle de conteneur
 - ▶ Docker
 - ▶ Podman

On ne met plus à jour les serveurs, mais on change le modèle et on déploie à nouveau

Outils de provisionnement d'infrastructure

La création et le provisionnement d'infrastructure ne sont plus décrits en utilisant un langage générique

- impose la structure de la configuration (dans le bon sens)
- décrit *ce qu'on veut*, pas *comment on fait pour l'avoir*
- Terraform/OpenTofu

```
resource "openstack_compute_instance_v2" "test-server" {  
    # ...  
}
```

Outils de gestion de configuration

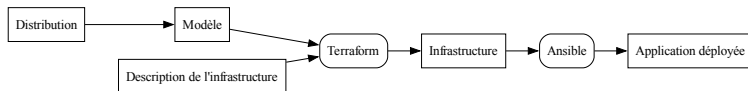
Une qu'on a l'infrastructure, on peut utiliser ces outils pour finir la configuration

- Ansible
- Chef
- Puppet

On décrit à nouveau **ce qu'on veut**, pas **comment**

- l'outil se charge d'appliquer le *delta* entre l'existant et le souhaité

En pratique



Plan du cours

1 Semaine 2

- Un mot sur la production logicielle
- Du *Dev et Ops* au *DevOps*
- Infrastructure as Code
- Préambule à Terraform/OpenTofu

Terraform

- Produit par HashiCorp (comme Vagrant)
- Fork fin 2023 par la Linux Fondation : OpenTofu
 - ▶ maintient d'une vraie licence libre

Principe

- 1 Choisir un *provider* cloud
- 2 Décrire une infrastructure sous la forme d'un fichier listant des ressources (modèle, matériel, réseau...)
- 3 Laisser terraform utiliser l'API du fournisseur cloud pour que l'infrastructure déployée corresponde à la description
- 4 Effectuer la personnalisation initiale des VM
- 5 Déployer l'application (outils de gestion de configuration...)
- 6 Répéter pour mettre à jour

Personnalisation initiale : problématique

- Plusieurs distribution possibles pour les modèles (debian, ubuntu, centos...)
- Plusieurs fournisseurs cloud (OVHcloud, AWS, Azure, Openstack, Proxmox...)
- Comment rendre homogène la personnalisation initiale ?
 - ▶ création d'utilisateur sur la VM
 - ▶ déploiement de la clé SSH pour la connexion
 - ▶ configuration d'éléments spécifiques
 - ★ à un opérateur cloud
 - ★ à une distribution

Standard *de-facto* : cloud-init

Poussé à l'origine par Canonical pour Ubuntu, disponible pour la quasi totalité des distributions et des fournisseurs cloud

- construit une configuration à partir de plusieurs sources et l'applique au démarrage de l'instance
 - ▶ Configuration de base : fournie par la distribution
 - ▶ *Vendor data* : fournie par l'opérateur cloud
 - ▶ *User data* : fournie par l'utilisateur final

Configuration de base

Donnée dans `/etc/cloud` ou dans les paramètres du noyau

- mise en place par la distribution (ou le fournisseur du modèle)
 - ▶ dans les versions *cloud* de celles-ci : <https://cloud.debian.org> par exemple
 - ▶ décide des modules cloud-init à charger et de leur configuration par défaut

Vendor data et User data

Configuration Fournie par une *Source de donnée* qui dépend de l'opérateur cloud.

Peuvent être de différents types (scripts, cloud-config. . .)

Format cloud-config

Le format le plus simple à utiliser

- YAML
- Définit des variables qui affectent le comportement des **modules** de cloud-init

Exemple

Utilise le module *Set Passwords* pour configurer le mot de passe de l'utilisateur par défaut

```
#cloud-config  
password: toto  
chpasswd:  
    expire: False
```